

Курс: Углубленное машинное обучение и искусственный интеллект в Python

Практическое задание: Сверточные нейронные сети (CNN) для классификации изображений

Цель работы: Научиться проектировать и обучать сверточные нейронные сети (CNN) для задачи классификации изображений, подготавливать тензорные данные и оценивать качество работы модели на тестовой выборке.

Шаг 1. Загрузка и подготовка данных

В качестве источника данных используется классический датасет **CIFAR-10**, загруженный с платформы Hugging Face. Выполните загрузку и первичную предобработку:

1. Загрузите датасет с помощью библиотеки datasets.
2. Разделите данные на обучающую (train) и тестовую (test) выборки (используйте уже имеющееся в датасете разбиение).
3. Выполните нормализацию интенсивности пикселей изображений, приведя их из диапазона [0, 255] к диапазону [0, 1].

```
from datasets import load_dataset
import tensorflow as tf
```

```
# Загрузка датасета CIFAR-10
dataset = load_dataset("cifar10")
```

```
# Выполните нормализацию и подготовку тензоров для обучения
```

Шаг 2. Проектирование архитектуры CNN

Используя инструмент `tf.keras.Sequential`, постройте сверточную нейросеть следующей конфигурации:

- **Сверточный блок:** Последовательно добавьте 2–3 блока, каждый из которых

состоит из слоев Conv2D (с активацией ReLU) и MaxPooling2D.

- **Полносвязный блок:** Добавьте слой выравнивания Flatten, затем полносвязный слой Dense и слой регуляризации Dropout для предотвращения переобучения.
- **Выходной слой:** Полносвязный слой Dense(10, activation='softmax') для многоклассовой классификации на 10 классов.

```
model = tf.keras.Sequential([
    # Пример структуры первого блока (настройте параметры самостоятельно)
    tf.keras.layers.Input(shape=(32, 32, 3)),
    tf.keras.layers.Conv2D(32, (3, 3), activation='relu'),
    tf.keras.layers.MaxPooling2D((2, 2)),

    # Добавьте дополнительные сверточные блоки, Flatten, Dense и Dropout

    tf.keras.layers.Dense(10, activation='softmax')
])
```

Шаг 3. Компиляция и обучение модели

Настройте процесс обучения и запустите его:

1. Скомпилируйте модель, используя оптимизатор `optimizer='adam'` и функцию потерь `loss='sparse_categorical_crossentropy'`. В качестве метрики качества укажите `metrics=['accuracy']`.
2. Обучите нейросеть в течение **5–10 эпох** на обучающей выборке.

```
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

```
# Обучение модели
# model.fit(...)
```

Шаг 4. Оценка качества

Проверьте качество получившейся модели на отложенных данных:

- Выведите итоговую точность (Accuracy) модели на тестовом наборе данных.
- Сделайте краткий вывод о качестве классификации и о том, наблюдается ли у модели эффект переобучения при выбранном количестве эпох.